DESIGN PATTERNS & FRAMEWORKS

Template Method Design Pattern

Defining the **algorithm** once.
Customizing selected **steps**.

Theory & UML

Java Implementation

Class Discussion & Examples



Understand the pattern

Apply to real-world problems

Implement with Java

Where Are We?

Design Patterns We Have Covered

CREATIONAL
Patterns that deal with object creation

STRUCTURAL
Patterns that organize and compose objects

BEHAVIORAL
Patterns that define communication and responsibility

Pattern	Category	Main Idea
Singleton	Creational	Ensure only one instance exists
Factory Method	Creational	Delegate object creation
Facade	Structural	Provide a simplified interface
Proxy	Structural	Control access to an object
Decorator	Structural	Add responsibilities dynamically
Adapter	Structural	Make incompatible interfaces work together
Template Method	**Behavioral**	**Define the algorithm structure and allow selected steps to vary**

Quick Review Activity

Which Pattern Would You Use?

Give students **5–7** minutes to discuss the following.

Scenario A

An application must ensure that only one configuration manager exists.

Answer: **Singleton**

Scenario B

A payment system should create different payment processors depending on the selected payment type.

Answer: **Factory Method**

Scenario C

A complex home theatre system should be controlled through one simple interface.

Answer: **Facade**

Scenario D

Additional features such as premium access and themes should be added to a user profile dynamically.

Answer: **Decorator**

Scenario E

An existing legacy payment service has an incompatible interface.

Answer: **Adapter**

Scenario F

Several classes follow the same process, but some individual steps are implemented differently.

Answer: **Template Method**

Key Transition: Today we focus on Scenario F.

5 What Is the Template Method Pattern?

Template Method Design Pattern

> Defines the skeleton of an algorithm in a superclass but allows subclasses to override specific steps without changing the algorithm's overall structure.

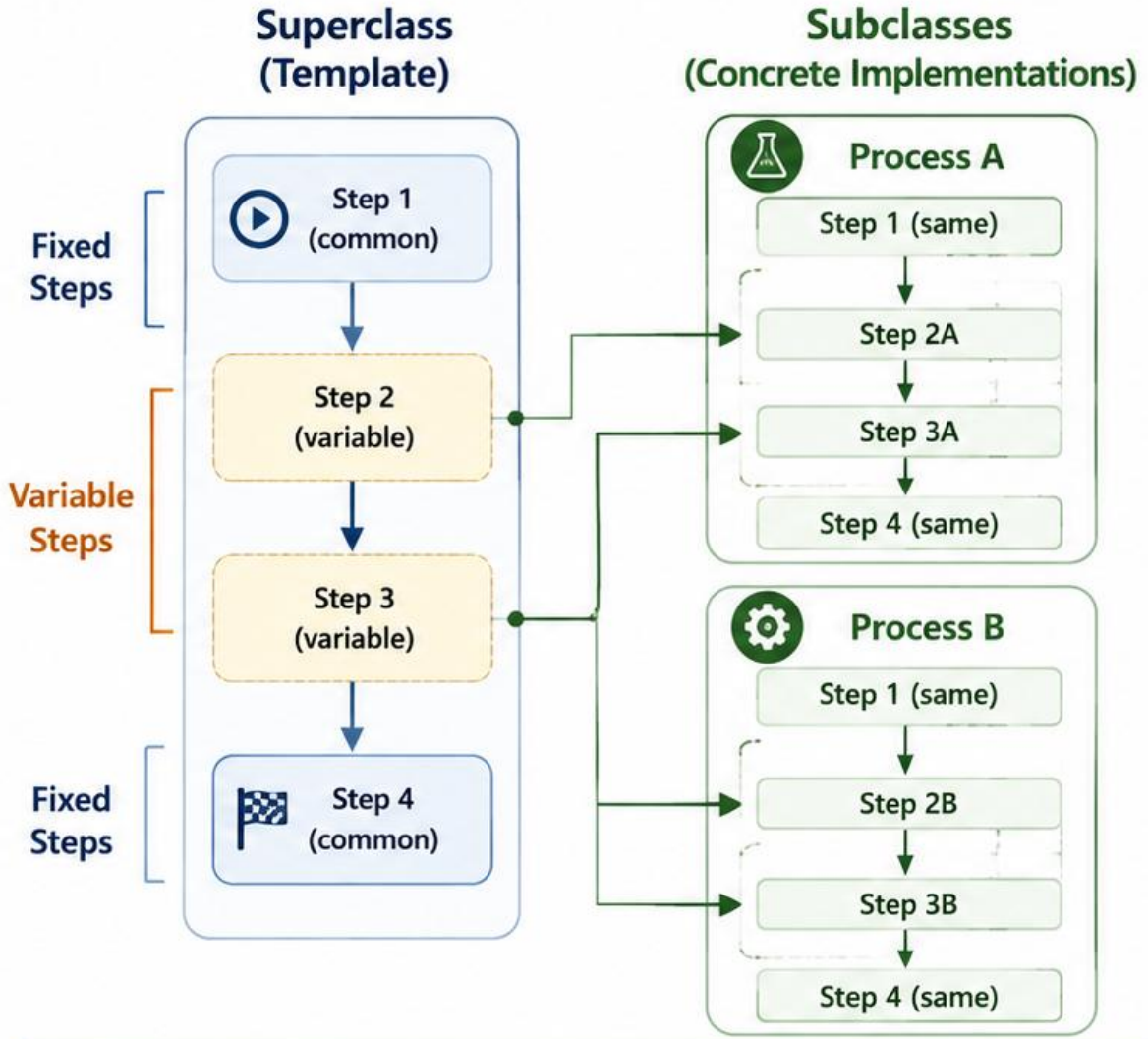
Key Idea

Superclass controls:

- The sequence
- The overall workflow
- Common operations

Subclasses control:

- Variable steps
- Specific implementations
- Domain-specific behaviour



Result: Reuse common code, reduce duplication, and ensure a consistent algorithm structure.

Template Method Pattern

The Core Concept

Define the skeleton of an algorithm in the superclass and let subclasses redefine certain steps.

```
abstract class Process {
  final void execute() {
    step1();
    step2();
    step3();
    step4();
  }
  // steps to be implemented
}
```



The Algorithm Is Fixed

Template Method



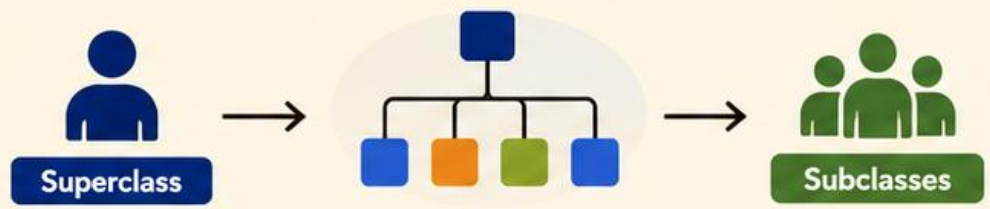
The structure and order of steps are controlled by the superclass.
Subclasses only provide details.

Important Principle

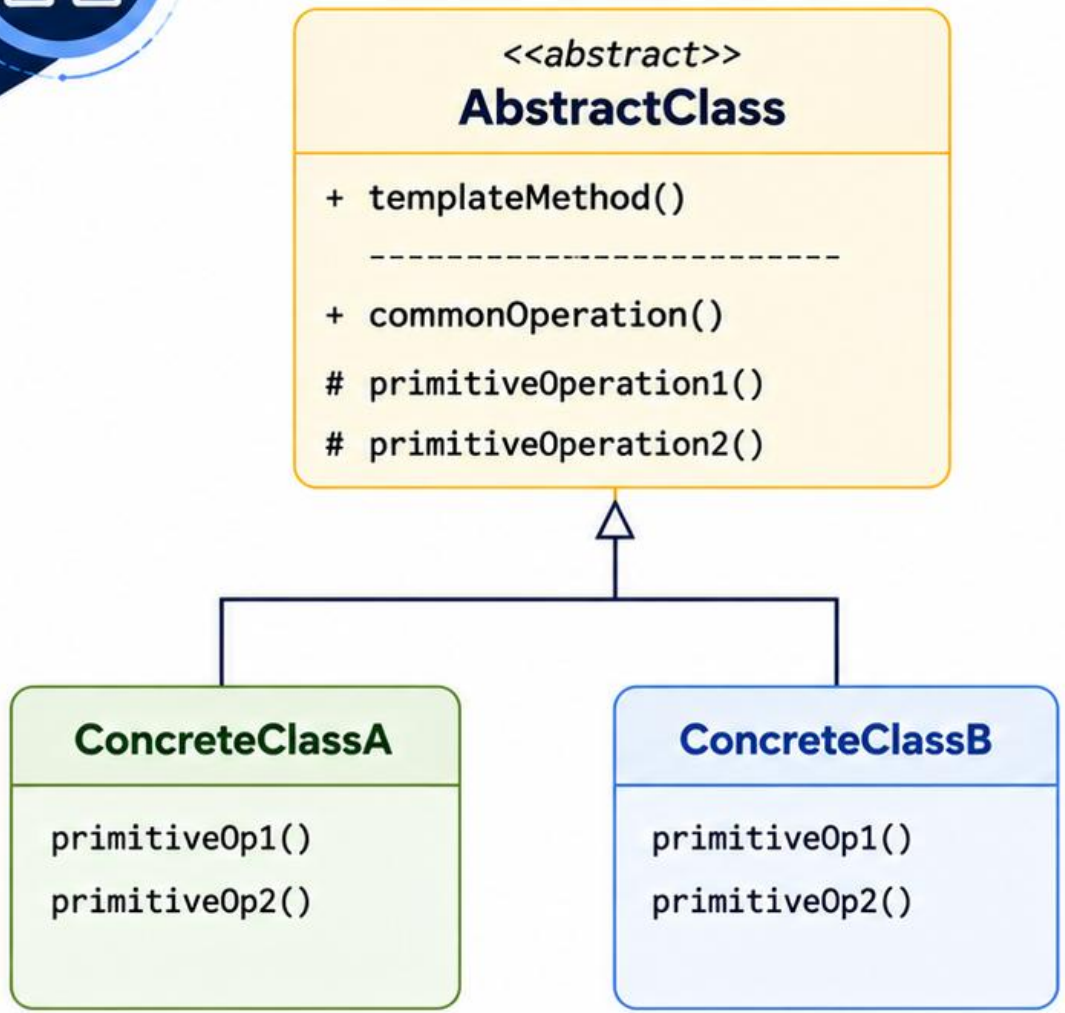
- The subclass can **change how a step is performed.**
- It should **not change** the overall algorithm **sequence.**

Think About It

Who should control the workflow: the superclass or each subclass? Why?



Generic UML Structure



Important Roles

Abstract Class

- Defines the template method.
- Defines the sequence of operations.
- Implements common steps.
- Declares abstract or overridable steps.

Concrete Classes

- Implement the variable steps.
- Customize the algorithm where necessary.

Real-World Examples Using the Same Structure

SelfDrivingVehicle
The algorithm for driving is defined in the base class.
Steps like “detect obstacles” or “apply brakes” are implemented by subclasses.

PastaDish
The recipe steps are defined in the base class.
Steps like “boil pasta” or “add sauce” are implemented by subclasses.

Template Method Pattern: Define the skeleton of an algorithm in an operation, deferring some steps to subclasses.

6 Java Implementation

Abstract Class

Template Method

Hook Methods

Common Operations

```java
public abstract class DataProcessor {

    // Template Method
    public final void processData() {
        loadData();
        cleanData();
        analyzeData();
        generateReport();
    }

    private void loadData() {
        System.out.println("Loading data");
    }

    protected abstract void cleanData();

    protected abstract void analyzeData();

    private void generateReport() {
        System.out.println("Generating report");
    }
}
```

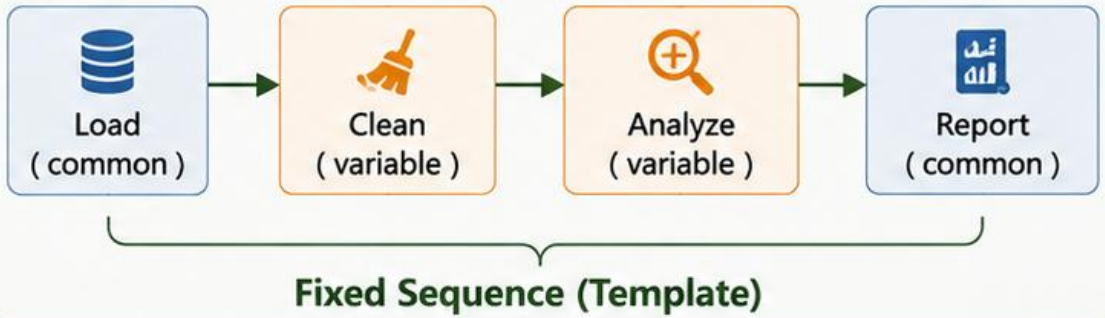
Discussion Point

Why is `processData()` declared as `final`?

Expected Answer

To **prevent subclasses** from **changing the sequence of the algorithm**.

They can customize `cleanData()` and `analyzeData()`, but **cannot rearrange**:



Key Takeaway

The final template method enforces a consistent workflow, promotes code reuse, and ensures algorithm integrity.

Concrete Implementations

Subclasses implement the **variable steps** while the superclass **defines the algorithm**.

CSVDataProcessor.java

```java
public class CSVDataProcessor
   extends DataProcessor {

  @Override
  protected void cleanData() {
    System.out.println("Cleaning CSV data");
  }

  @Override
  protected void analyzeData() {
    System.out.println("Analyzing CSV data");
  }
}
```

DatabaseDataProcessor.java

```java
public class DatabaseDataProcessor
   extends DataProcessor {

  @Override
  protected void cleanData() {
    System.out.println("Cleaning database records");
  }

  @Override
  protected void analyzeData() {
    System.out.println("Analyzing database records");
  }
}
```

Main.java

```java
public class Main {

  public static void main(String[] args) {

    DataProcessor csv =
      new CSVDataProcessor();

    DataProcessor database =
      new DatabaseDataProcessor();

    csv.processData();
    System.out.println();
    database.processData();
  }
}
```

Ask Students

Which method controls the complete algorithm?

Answer: `processData()`

Program Output

```
Cleaning CSV data              // from CSVDataProcessor.cleanData()
Analyzing CSV data             // from CSVDataProcessor.analyzeData()

Cleaning database records      // from DatabaseDataProcessor.cleanData()
Analyzing database records     // from DatabaseDataProcessor.analyzeData()
```

OOP CONCEPTS YOU MUST KNOW
TO UNDERSTAND THE TEMPLATE METHOD PATTERN

1 CLASSES & OBJECTS

A class is a blueprint. An object is an instance of a class.

```
DataProcessor p =
new CSVDataProcessor();
```

2 INHERITANCE

Subclasses inherit fields and methods from the superclass.

```
class CSVDataProcessor
  extends DataProcessor { }
```

3 ABSTRACT CLASS

Provides a common structure. Cannot be instantiated.

```
public abstract class
DataProcessor { ... }
```

4 ABSTRACT METHOD

Declares methods that subclasses must implement.

```
protected abstract void
cleanData();
```

5 METHOD OVERRIDING

Subclasses provide their own implementation for inherited methods.

```
@Override
protected void cleanData() {
   // custom implementation
}
```

6 ACCESS MODIFIERS

Control visibility and what subclasses can access/override.

```
public    - everywhere
protected - to subclasses
private   - within same class
```

7 FINAL KEYWORD

Prevents subclasses from changing the algorithm structure.

```
public final void
processData() { ... }
```

8 POLYMORPHISM

A superclass reference can refer to different subclass objects.

```
DataProcessor p =
new CSVDataProcessor();
p.processData();
```

TEMPLATE METHOD PATTERN – THE IDEA

The superclass defines the skeleton (algorithm).

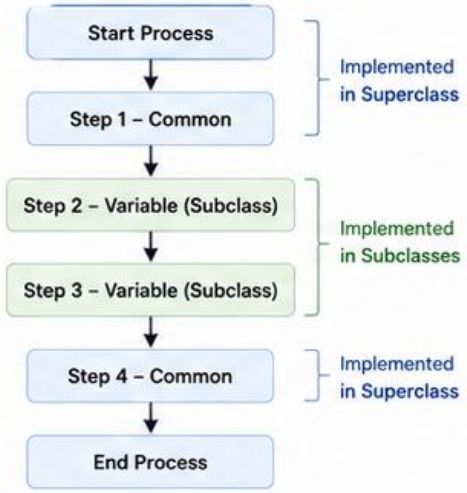
- It controls the sequence of steps.
- It implements the common steps.
- It declares the variable steps as abstract.

Subclasses implement the variable steps.

- They customize specific steps.
- They cannot change the order of steps.
- This promotes code reuse and consistency.

Key Point:
Subclasses can change how a step is performed, but they cannot change the overall algorithm sequence.

EXAMPLE ALGORITHM FLOW



SAMPLE CODE STRUCTURE

```
// Abstract Class
abstract class DataProcessor {
   public final void processData() {   // Template Method
       loadData();                      // common
       cleanData();                     // variable
       analyzeData();                   // variable
       generateReport();                // common
   }
   private void loadData() {
       System.out.println("Loading data");
   }
   protected abstract void cleanData();
   protected abstract void analyzeData();
   private void generateReport() {
       System.out.println("Generating report");
   }
}
```

```
// Concrete Class A
class CSVDataProcessor extends DataProcessor {
   @Override
   protected void cleanData() {
       System.out.println("Cleaning CSV data");
   }
   @Override
   protected void analyzeData() {
       System.out.println("Analyzing CSV data");
   }
}
```

```
// Concrete Class B
class DBDataProcessor extends DataProcessor {
   @Override
   protected void cleanData() {
       System.out.println("Cleaning DB data");
   }
   @Override
   protected void analyzeData() {
       System.out.println("Analyzing DB data");
   }
}
```

WHY IMPORTANT?

- Controls the flow of the algorithm.
- Centralizes common operations.
- Reduces code duplication.
- Promotes code reusability.
- Easy to maintain and extend.

CHALLENGES

- The algorithm flow cannot be changed by subclasses.
- May become rigid if the sequence needs frequent changes.
- Too many subclasses can make the design harder to maintain.

TEMPLATE METHOD vs OTHER PATTERNS

Pattern	Focus
Template Method (Behavioral)	How an algorithm is executed (sequence of steps).
Factory Method (Creational)	Which object to create.
Decorator (Structural)	Add responsibilities to an object.
Strategy (Behavioral)	Choose one algorithm among many at runtime.

SUMMARY

To understand the Template Method pattern, you must be comfortable with Inheritance, Abstract Classes, Abstract Methods, Method Overriding, Access Modifiers, the final keyword, and Polymorphism.



7 Why Not Just Override Everything?

Without Template Method

CSV Processor

- Load
- Clean CSV
- Analyze CSV
- Generate Report

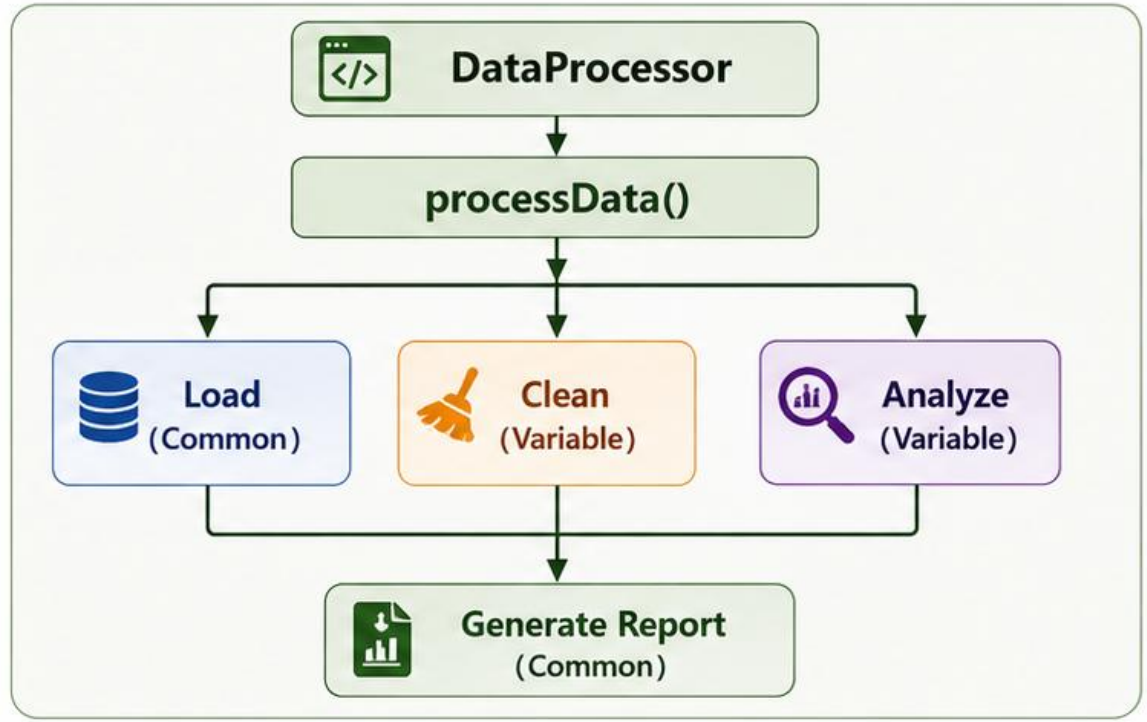
Database Processor

- Load
- Clean Database
- Analyze Database
- Generate Report

- ✗ Repeated algorithm
- ✗ Possible inconsistency
- ✗ Difficult to maintain



With Template Method



Benefits

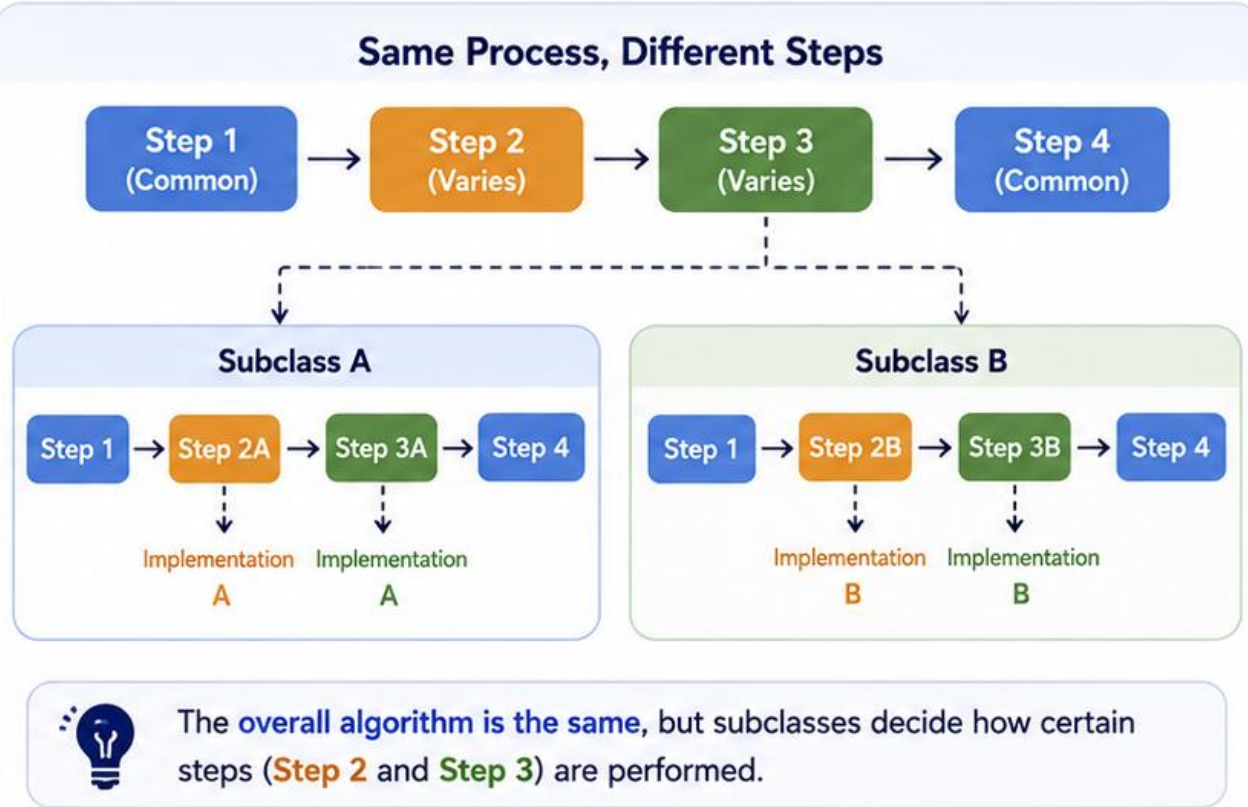
- Centralized common operations
- Reduced duplication
- Code reuse and easier maintenance

Key Takeaway: Template Method defines the algorithm structure once and lets subclasses focus only on what varies.

When Should We Use It?

Use Template Method when:

- Multiple classes follow the **same sequence** of operations.
- Some steps are **common**.
- Some steps **vary** between subclasses.
- You want to **avoid duplicating** the complete algorithm.
- The sequence of execution should be **controlled centrally**.
- You are developing **frameworks** or **reusable process structures**.



The **overall algorithm is the same**, but subclasses decide how certain steps (**Step 2** and **Step 3**) are performed.

Simple Formula

Same Process + Different Steps = Consider Template Method

Discussion Case 1: University Examination Process

Scenario

The university conducts different types of examinations:

- Theory Examination
- Programming Examination
- Practical Laboratory Examination

All examinations follow a common process:

1. Register Students
2. Prepare Examination
3. Conduct Examination
4. Evaluate
5. Publish Results

Key Idea

The overall process (**sequence**) is the same, but some steps **vary** based on the type of examination.

However, some steps differ.

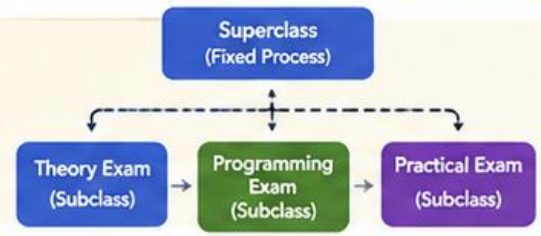
	Theory Examination	Programming Examination	Practical Examination
1 Register Students	Register all eligible students.	Register all eligible students.	Register all eligible students.
2 Prepare Examination	Prepare question paper.	Prepare programming problems.	Prepare laboratory tasks.
3 Conduct Examination	Conduct written examination.	Conduct examination using an IDE.	Conduct laboratory examination.
4 Evaluate	Manual evaluation.	Automated + manual evaluation.	Evaluate practical performance.
5 Publish Results	Publish results to students.	Publish results to students.	Publish results to students.

Takeaway

The **overall algorithm** (process) is fixed across all examination types, but the **implementation** of certain steps is customized.

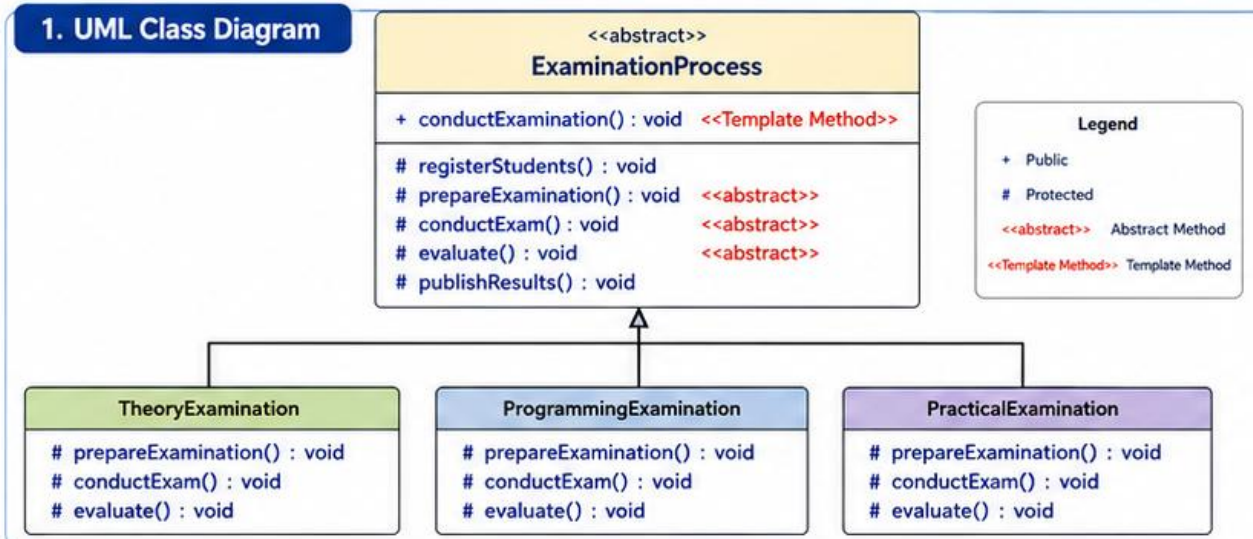
This is exactly the idea behind the

Template Method Pattern



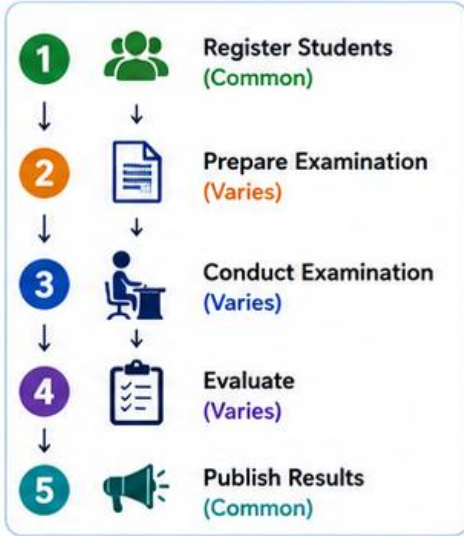
Discussion Case 1: University Examination Process – Template Method Pattern

1. UML Class Diagram



2. Template Method (Fixed Algorithm)

```
public final void conductExamination() {
    registerStudents();          // Step 1: Common
    prepareExamination();        // Step 2: Varies
    conductExam();               // Step 3: Varies
    evaluate();                  // Step 4: Varies
    publishResults();            // Step 5: Common
}
```



3. Java Code – Implementations

```
abstract class ExaminationProcess {
    // Template Method
    public final void conductExamination() {
        registerStudents();
        prepareExamination();
        conductExam();
        evaluate();
        publishResults();
    }
    // Common Step
    protected void registerStudents() {
        System.out.println("Registering eligible students...");
    }
    // Steps that vary
    protected abstract void prepareExamination();
    protected abstract void conductExam();
    protected abstract void evaluate();

    // Common Step
    protected void publishResults() {
        System.out.println("Publishing examination results...");
    }
}
```

TheoryExamination

```
class TheoryExamination extends ExaminationProcess {

    @Override
    protected void prepareExamination() {
        System.out.println("Preparing question paper...");
    }

    @Override
    protected void conductExam() {
        System.out.println("Conducting written " +
                           "examination...");
    }

    @Override
    protected void evaluate() {
        System.out.println("Performing manual " +
                           "evaluation...");
    }
}
```

ProgrammingExamination

```
class ProgrammingExamination extends ExaminationProcess {

    @Override
    protected void prepareExamination() {
        System.out.println("Preparing programming " +
                           "problems...");
    }

    @Override
    protected void conductExam() {
        System.out.println("Conducting examination " +
                           "using an IDE...");
    }

    @Override
    protected void evaluate() {
        System.out.println("Performing automated " +
                           "and manual evaluation...");
    }
}
```

PracticalExamination

```
class PracticalExamination extends ExaminationProcess {

    @Override
    protected void prepareExamination() {
        System.out.println("Preparing laboratory tasks...");
    }

    @Override
    protected void conductExam() {
        System.out.println("Conducting laboratory " +
                           "examination...");
    }

    @Override
    protected void evaluate() {
        System.out.println("Evaluating practical " +
                           "performance...");
    }
}
```

4. Main Class (Client)

```
public class UniversityExaminationDemo {
    public static void main(String[] args) {
        ExaminationProcess theoryExam = new TheoryExamination();
        ExaminationProcess programmingExam = new ProgrammingExamination();
        ExaminationProcess practicalExam = new PracticalExamination();

        System.out.println("\n=== THEORY EXAMINATION ===");
        theoryExat.conductExamination();

        System.out.println("\n=== PROGRAMMING EXAMINATION ===");
        programmingExam.conductExamination();

        System.out.println("\n=== PRACTICAL EXAMINATION ===");
        practicalExam.conductExamination();
    }
}
```

5. Sample Output

```
=== THEORY EXAMINATION ===
Registering eligible students...
Preparing question paper...
Conducting written examinatio....
Performing manual evaluation...
Publishing examination results...
=== PROGRAMMING EXAMINATION ===
Registering eligible students...
Preparing programming problems...
Conducting examination using an IDE...
Performing automated and manual evaluation...
Publishing examination results...
```

```
=== PRACTICAL EXAMINATION ===
Registering eligible students...
Preparing laboratory tasks...
Conducting laboratory examination...
Evaluating practical performance...
Publishing examination results...
```

Key Takeaway

- The overall algorithm (sequence of steps) is fixed in the superclass.
- Subclasses provide their own implementation for the varying steps.
- This avoids code duplication and promotes reusability.
- Ideal for frameworks and processes with a stable workflow.

Simple Formula: Same Process + Different Steps = Consider Template Method

8 Discussion Case 2: Online Shopping Order Processing

Scenario

An e-commerce system processes different order types:

- Standard Order
- Premium Order
- International Order

All orders follow this basic process:

1. Validate Order
2. Calculate Price
3. Process Payment
4. Prepare Shipment
5. Send Notification

However:

Standard Order
- Normal delivery
- Standard packaging

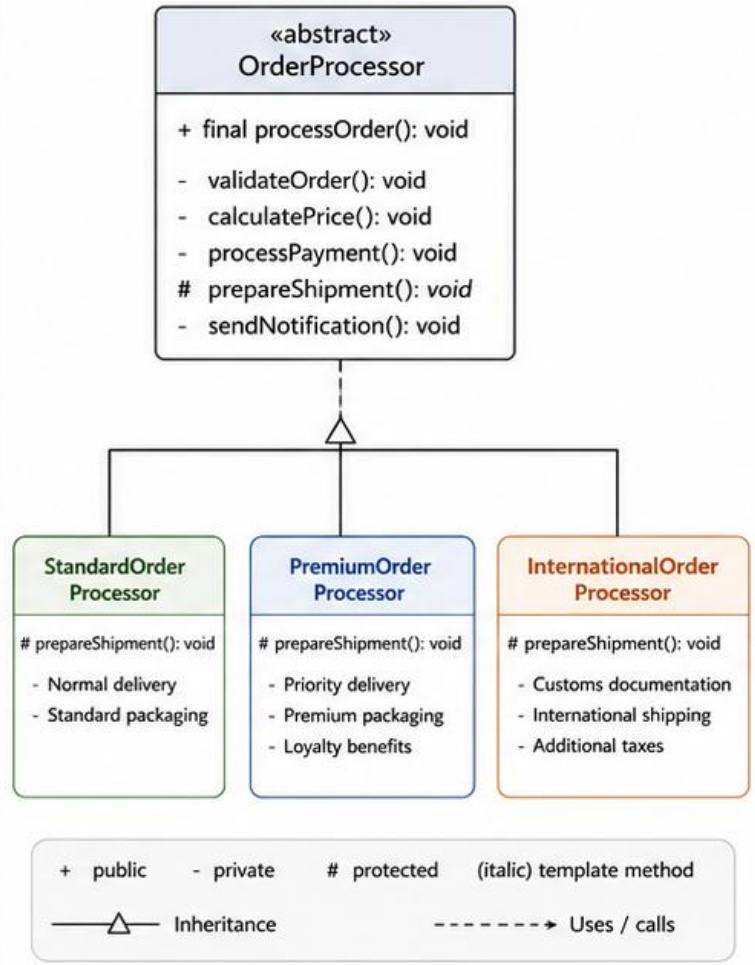
Premium Order
- Priority delivery
- Premium packaging
- Loyalty benefits

International Order
- Customs documentation
- International shipping
- Additional taxes

Discussion Questions:

1. Which steps are common across all order types?
2. Which steps vary between order types?
3. How can we use the Template Method Pattern here?

UML Class Diagram



Java Implementation

```java
// Abstract Class
public abstract class OrderProcessor {
    public final void processOrder() {
        validateOrder();
        calculatePrice();
        processPayment();
        prepareShipment();   // Varies
        sendNotification();
    }

    private void validateOrder() { /* common implementation */ }
    private void calculatePrice() { /* common implementation */ }
    private void processPayment() { /* common implementation */ }
    private void sendNotification() { /* common implementation */ }

    protected abstract void prepareShipment(); // variable step
}
// Concrete Implementations
class StandardOrderProcessor extends OrderProcessor {
    @Override
    protected void prepareShipment() {
        // Normal delivery
        // Standard packaging
    }
}
class PremiumOrderProcessor extends OrderProcessor {
    @Override
    protected void prepareShipment() {
        // Priority delivery
        // Premium packaging
        // Loyalty benefits
    }
}
class InternationalOrderProcessor extends OrderProcessor {
    @Override
    protected void prepareShipment() {
        // Customs documentation
        // International shipping
        // Additional taxes
    }
}
```

Key Takeaway: The Template Method Pattern lets us **define the order processing workflow once** in the superclass, while allowing each order type to **customize only the shipment preparation step**.

TEMPLATE METHOD DESIGN PATTERN

Define the algorithm structure once, allow subclasses to customize selected steps.

BEHAVIORAL

1. OVERVIEW

The Template Method pattern defines the skeleton of an algorithm in a superclass but allows subclasses to override specific steps without changing the overall structure.

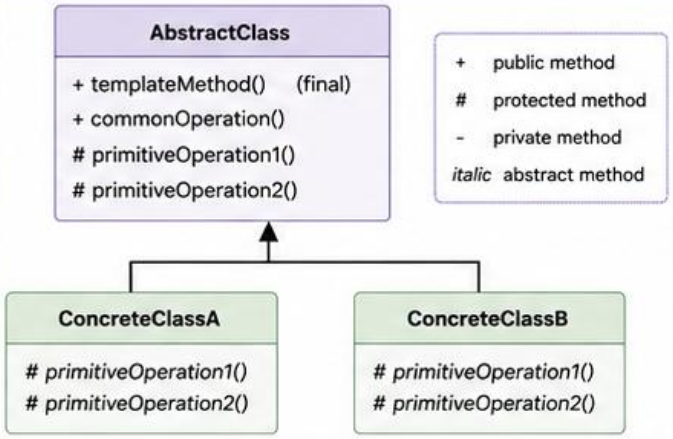
Superclass controls:
the sequence and the common steps of the algorithm.

Subclasses control:
the implementation of variable steps.

2. WHEN TO USE

- Multiple classes follow the same sequence of operations.
- Some steps are common and some vary.
- You want to avoid code duplication.
- You want to control the algorithm structure centrally.
- Useful in framework and library development.

3. UML STRUCTURE



templateMethod() defines the algorithm skeleton.
Subclasses implement the primitive operations.
The overall algorithm structure remains unchanged.

4. ALGORITHM FLOW (TEMPLATE METHOD)

Step 1 (Common) → Step 2 (Variable) → Step 3 (Variable) → Step 4 (Common)

Subclasses can change how a step is performed, but cannot change the order of steps.

5. JAVA EXAMPLE

```java
// Abstract Class
abstract class DataProcessor {
    public final void processData() {
        loadData();
        cleanData();        // variable step
        analyzeData();      // variable step
        generateReport();
    }

    private void loadData() {
        System.out.println("Loading data");
    }

    protected abstract void cleanData();
    protected abstract void analyzeData();

    private void generateReport() {
        System.out.println("Generating report");
    }
}
```

```java
// Concrete Class A
class CSVProcessor extends DataProcessor {
    protected void cleanData() {
        System.out.println("Cleaning CSV data");
    }
    protected void analyzeData() {
        System.out.println("Analyzing CSV data");
    }
}
```

```java
// Concrete Class B
class DBProcessor extends DataProcessor {
    protected void cleanData() {
        System.out.println("Cleaning DB data");
    }
    protected void analyzeData() {
        System.out.println("Analyzing DB data");
    }
}
```

```java
// Execution
public class Main {
    public static void main(String[] args) {
        DataProcessor p1 = new CSVProcessor();
        p1.processData();
        System.out.println("--------------------");
        DatsProcessor p2 = new DBProcessor();
        p2.processData();
    }
}
```

```
// Output (Example)
Loading data
Cleaning CSV data
Analyzing CSV data
Generating report
--------------------
Loading data
Cleaning DB data
Analyzing DB data
Generating report
```

6. ADVANTAGES

- Controls the flow of the algorithm.
- Centralizes common operations.
- Reduces code duplication.
- Promotes code reusability.
- Easy to maintain and extend.

7. CHALLENGES

- The algorithm flow cannot be changed by subclasses.
- May become rigid if the sequence needs frequent changes.
- Too many subclasses can make the design harder to maintain.

8. TEMPLATE METHOD vs STRATEGY

Aspect	Template Method	Strategy
Intent	Define skeleton of an algorithm	Encapsulate interchangeable algorithms
Focus	Sequence of steps	Algorithm implementation
Inheritance	Uses inheritance (extends class)	Uses composition (has a strategy)
When to use	When steps are same in order, some vary	When the entire algorithm can vary

9. KEY TAKEAWAYS

- Template Method is a Behavioral Design Pattern.
- It defines the algorithm structure in a superclass.
- Subclasses customize specific steps.
- It ensures consistent execution flow while allowing flexibility.
- Widely used in frameworks (e.g., JUnit, Servlet, Spring).

Think: The Template Method pattern is about controlling the "what" and "when" in the superclass, and letting subclasses decide the "how" for specific steps.